INFORME DE LA PRUEBA PARACTICA UNIDADA 2

GRUPO M

INTEGRANTES : Alexander Guaman
Mateo Muyulema
Alejandro Villacres

CARRERA: Software **PARALELO:** "B"

ASIGNATURA: Algoritmos y lógica de programación

DOCENTE : Ing. Jose Caiza

NIVEL: 1

FECHA: 14/04/2026

REPOSITORIO GITHUB : https://github.com/mateomuyulema2007-cmyk/Prueba-Unidad-2

TEMA: Evaluación practica

1. OBJETIVO GENERAL

Desarrollar un sistema integral de gestión y evaluación de proyectos estudiantiles en C++ mediante el paradigma de Programación Orientada a Objetos y algoritmos de procesamiento manual, con el fin de automatizar el cálculo de calificaciones académicas y reforzar el dominio de terminología técnica a través de un módulo gamificado.

OBJETIVOS ESPECIFICOS

- Implementar una arquitectura de software robusta utilizando clases, constructores y encapsulamiento (métodos Getters y Setters), garantizando la integridad de los datos y la modularidad del código fuente.
- Aplicar algoritmos de ordenamiento y búsqueda manuales (método de la burbuja y búsqueda secuencial) para la manipulación de registros estudiantiles, cumpliendo con los requisitos de lógica algorítmica sin depender de librerías automáticas.
- Integrar un módulo de bonificación funcional basado en el juego del "Ahorcado", que relacione el éxito en el juego con la mejora de la calificación final (Innovación), demostrando la interconexión entre diferentes módulos del sistema.
- Evidenciar el ciclo de vida del desarrollo mediante el uso de GitHub Desktop, documentando el progreso del proyecto a través de commits organizados y un archivo README técnico para la defensa del software.

2. PROBLEMA A RESOLVER

Contexto del caso: Registrar grupos, avances, retrasos, criterios, penalizaciones, bonificaciones y nota final.
Clases mínimas sugeridas: GrupoProyecto y Evaluacion

Requerimientos funcionales obligatorios
Registrar grupos con código, integrantes, tema, porcentaje de avance, retrasos y observaciones.
Registrar evaluaciones parciales con criterios ponderados y validación de suma total.
Calcular nota final con penalización por retrasos y bonificación por innovación.
Buscar por código y tema, ordenar manualmente por nota final y menor número de observaciones.

Emitir matrices simplificadas, mejor promedio y criterio de menor desempeño.
Integración obligatoria con Ahorcado: Construir un Ahorcado con criterios, defensa, avances, innovación y evaluación.

3. ANALISIS DEL PROBLEMA

La Universidad requiere una herramienta que permita a los docentes gestionar la evaluación de proyectos . El reto principal no es solo almacenar datos, sino garantizar que el cálculo de la nota final sea justo, manejando penalizaciones por retrasos y ofreciendo incentivos mediante un módulo de gamificación (Ahorcado). Además, se debe asegurar la integridad de la información mediante validaciones estrictas de tipo de dato.

3.1 VARIABLES DE ENTRADA PROCESO Y SALIDA

1. Entradas

Opciones numéricas para moverse por los menús.

Datos del proyecto: Código, integrantes, tema, % de avance, cantidad de retrasos y observaciones.

Calificaciones: Notas de funcionalidad, defensa, estructura e innovación.

Para buscar: El código o tema del grupo a consultar.

Ahorcado: Nivel de dificultad y las letras ingresadas para intentar adivinar la palabra.

2. Procesos:

Validación matemática: Comprobar que la suma de notas base no supere los 10 puntos.

Cálculo de la Nota Final: Sumar las notas, agregar la innovación y restablecer 0.5 puntos por cada retraso.

Ordenamiento y Búsqueda: Ordenar a los grupos de mayor a menor nota y buscar grupos específicos comparando el texto.

Cálculos estadísticos: Calcula el promedio de toda la clase usando recursividad y encuentra la calificación más baja de todas.

Lógica del Ahorcado: Escoger una palabra al azar, revisar si la letra ingresada es correcta para revelar sus posiciones o reiniciar un intento si es incorrecta, y calcular los puntos ganados.

3. Salidas:

Alertas de éxito o error (ej. "Registro exitoso" o "Entrada inválida").

Reporte tipo tabla con los proyectos ordenados (Código, % Avance, Nota, Tema).

El promedio general de la clase y el criterio con peor desempeño.

Los datos exactos del grupo que se buscó.

En el juego: el progreso de la palabra (ej. A _ ANC _), intentos restantes, mensaje de "Ganaste/Perdiste" y el historial de partidas con puntajes.

3.2 ALGORITMO

1. **Inicio**
2. Definir y Cargar parámetros: Definir vocabulario técnico para el Ahorcado (DEFENSA, RUBRICA, INNOVACION, etc.), límites de calificación (0-10) y valor de penalización por retraso (0.5 puntos).
3. Inicializar: Estructuras de datos (arreglos de objetos de la clase GrupoProyecto) para almacenar los registros de los equipos y variables para el conteo de grupos registrados.
   Menú principal: Presentar un menú con las opciones disponibles:
   Registrar Grupo y Evaluación.
   Generar Reporte Ordenado (Bubble Sort).
   Calcular Estadísticas (Promedio Recursivo).
   Jugar Ahorcado (Bonificación por Innovación).
   Buscar Grupo por Código/Tema.
   Salir.
4. Procesar opción: Ejecutar la acción correspondiente según la selección del usuario:
   Si es "Registrar Grupo y Evaluación": * Solicitar identificación del grupo, tema y nombre de integrantes.
   Solicitar notas de Funcionalidad, Defensa y Estructura.
   Validar: Asegurar que las entradas sean numéricas (validarFloat) y que la suma no exceda 10.0.
   Solicitar número de retrasos acumulados.
   - Procesar: Calcular la nota final inicial (Suma de notas - [retrasos * 0.5]).
     Almacenar el objeto en el arreglo y aumentar el contador.
     Si es "Generar Reporte Ordenado": * Ejecutar algoritmo de ordenamiento manual (Burbuja) comparando las notas finales de mayor a menor.
     Mostrar en pantalla el listado consolidado (Código, % de Avance, Nota Final y Tema).
     Si es "Calcular Estadísticas": * Llamar a la función recursiva para sumar las notas de todos los grupos registrados.
     Calcular el promedio general dividiendo la suma para el total de grupos.
     Mostrar el rendimiento promedio del curso.
     Si es "Jugar Ahorcado": * Seleccionar un término técnico aleatorio del dominio de evaluación.
     Mostrar tablero inicial con guiones bajos.
     Ciclo de juego: Solicitar letras, validar que no se repitan, actualizar tablero y controlar intentos.
5. **Resultado:** Si el usuario gana, aplicar automáticamente un bono de +0.5 a la nota final del último grupo registrado. Si pierde, mostrar la palabra correcta.
   Si es "Buscar Grupo": * Solicitar criterio de búsqueda (código o tema).
   Realizar búsqueda secuencial en el arreglo.
   Si se encuentra, mostrar los detalles completos del proyecto y sus criterios de evaluación.
   Repetir: Volver al paso 4 hasta que la opción seleccionada sea "Salir".

6. **Fin**

3.3. Pseudocodigo Pseint

```
Algoritmo GestionProyectos
Definir codigo, tema, integrantes, observaciones, clave Como Cadena
```

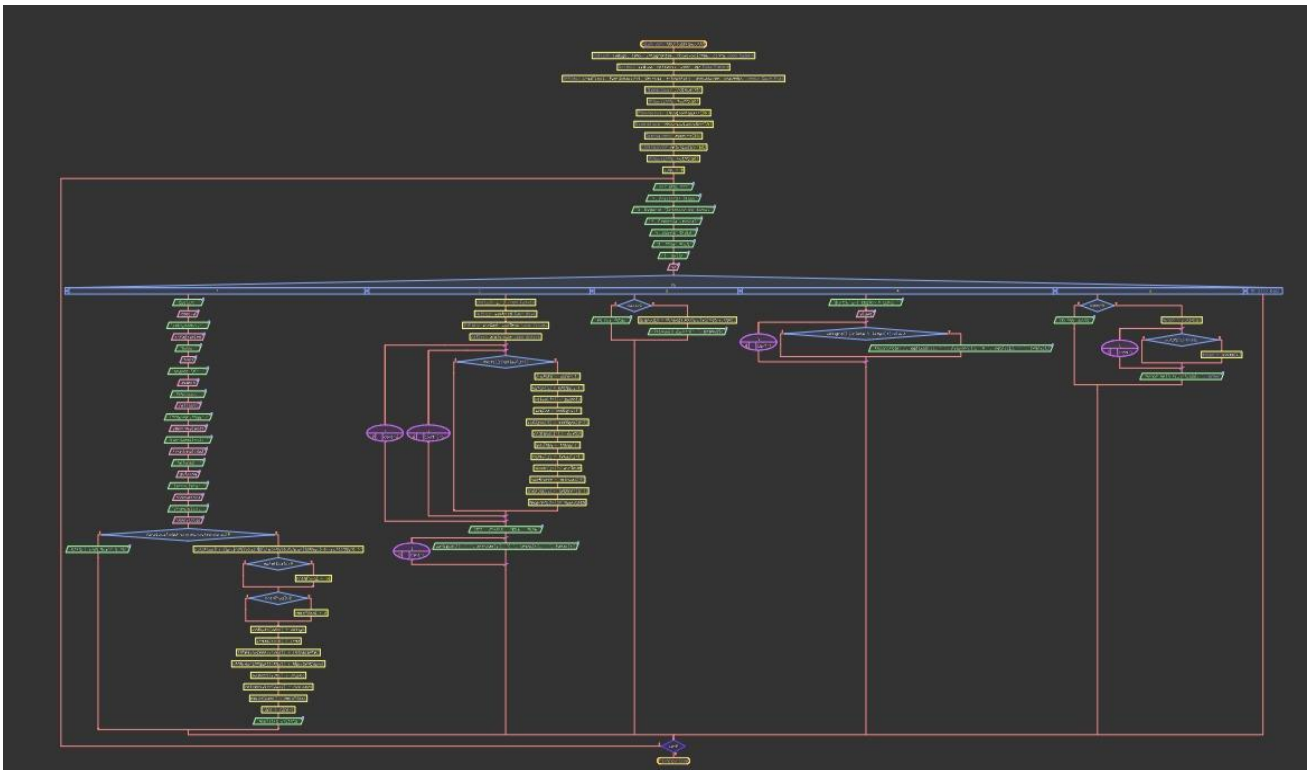
```
Definir avance, retrasos, cant, op Como Entero
Definir notaFinal, funcionalidad, defensa, estructura, innovacion, promedio, menor Como
Real
Dimensionar codigos(50)
Dimensionar temas(50)
Dimensionar integrantesArr(50)
Dimensionar observacionesArr(50)
Dimensionar avances(50)
Dimensionar retrasosArr(50)
Dimensionar notas(50)
cant <- 0
Repetir
Escribir '=== MENU ===='
Escribir '1. Registrador Grupo'
Escribir '2. Reporte (Ordenado por Nota)'
Escribir '3. Promedio General'Escribir'4
. Buscar Grupo'
Escribir '5. Menor Nota'Escribir'6
. Salir'
Leer op
Según op Hacer
1:
Escribir 'Codigo: '
Leer codigo
Escribir 'Integrantes: '
Leer integrantes
Escribir 'Tema: '
Leer tema
Escribir 'Avance (%): '
Leer avance
Escribir 'Retrasos: '
Leer retrasos
Escribir 'Observaciones: '
Leer observaciones
Escribir 'Funcionalidad: '
Leer funcionalidad
Escribir 'Defensa: '
Leer defensa
Escribir 'Estructura: '
Leer estructura
Escribir 'Innovacion: '
Leer innovacion
Si funcionalidad+defensa+estructura<=10 Entonces
notaFinal <- funcionalidad+defensa+estructura+innovacion-(retrasos*0.5)
Si notaFinal>10 Entonces
notaFinal <- 10
FinSi
Si notaFinal<0 Entonces
notaFinal <- 0
FinSi
codigos[cant] <- codigo
temas[cant] <- tema
integrantesArr[cant] <- integrantes
```

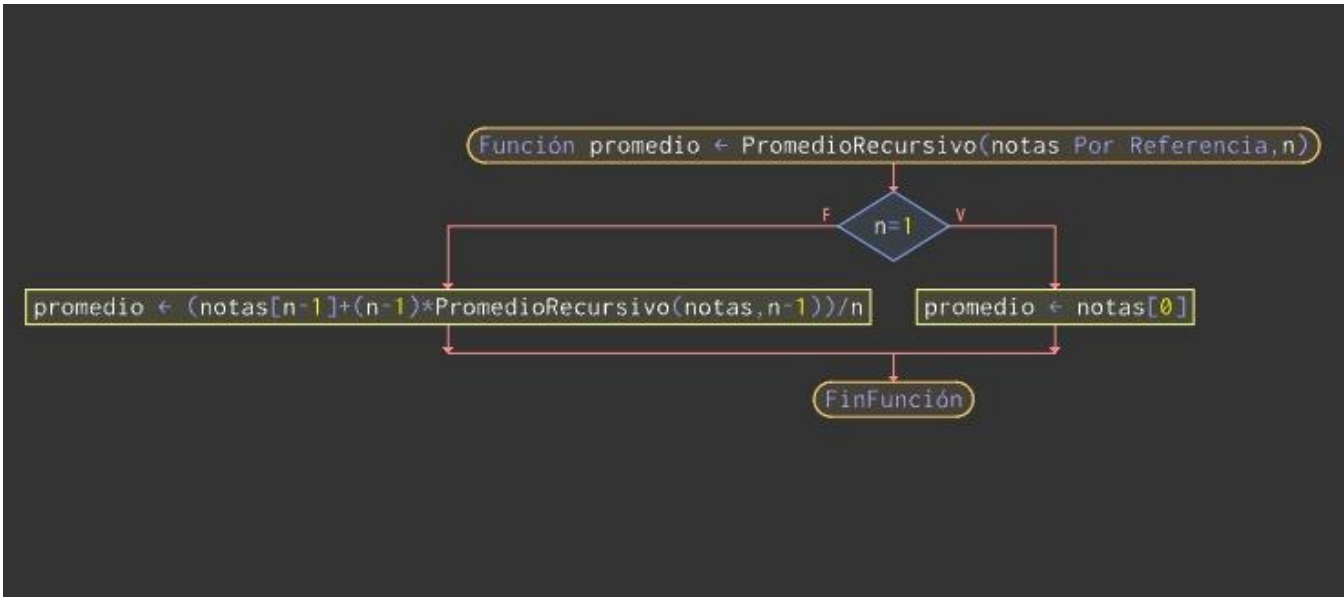
```
observacionesArr[cant] <- observaciones
avances[cant] <- avance
retrasosArr[cant] <- retrasos
notas[cant] <- notaFinal
cant <- cant+1
Escribir 'Registro exitoso'
SiNo
Escribir 'Error: suma mayor a 10'
FinSi
2:
Definir i, j Como Entero
Definir auxNota Como Real
Definir auxCod,auxTema Como Cadena
Definir auxAvance Como Entero
Para i<-0 Hasta cant-2 Hacer
Para j<-0 Hasta cant-2-i Hacer
Si notas[j]<notas[j+1] Entonces
auxNota <- notas[j]
notas[j] <- notas[j+1]
notas[j+1]<-auxNota
auxCod <- codigos[j]
codigos[j] <- codigos[j+1]
codigos[j+1]<-auxCod
auxTema <- temas[j]
temas[j] <- temas[j+1]
temas[j+1]<-auxTema
auxAvance <- avances[j]
avances[j] <- avances[j+1]
avances[j+1]<-auxAvance
FinSi
FinPara
FinPara
Escribir 'COD AVANCE NOTA TEMA'
Para i<-0 Hasta cant-1 Hacer
Escribir codigos[i], ' ', avances[i], '% ', notas[i], ' ', temas[i]
FinPara
3:
Si cant>0 Entonces
promedio <- PromedioRecursivo(notas,cant)
Escribir 'Promedio General: ', promedio
SiNo
Escribir 'No hay datos'
FinSi
4:
Escribir 'Buscar por codigo o tema: '
Leer clave
Para i<-0 Hasta cant-1 Hacer
Si codigos[i]=clave O temas[i]=clave Entonces
Escribir 'Encontrado: ', codigos[i], ' ', avances[i], '% ', notas[i], ' ', temas[i]
FinSi
FinPara
5:
Si cant>0 Entonces
```

```
menor <- notas[0]
Para i<-1 Hasta cant-1 Hacer
Si notas[i]<menor Entonces
menor <- notas[i]
FinSi
FinPara
Escribir 'Menor nota registrada: ', menor
SiNo
Escribir 'No hay datos'
FinSi
FinSegún
Hasta Que op=6
FinAlgoritmo
```

4. Función promedio <- PromedioRecursivo(notas Por Referencia,n)
   Si n=1 Entonces
   promedio <- notas[0]
   SiNo
   promedio <- (notas[n-1]+(n-1)*PromedioRecursivo(notas,n-1))/n
   FinSi
   FinFunción
5. Función Sin Título
6. FinFunción

3.4 Diagrama de Flujo

7. 



3.4. CODIGO

```
#include <iostream>
#include <string>
#include <vector>
#include <limits>
#include <ctime>
#include <algorithm>
using namespace std;

struct HistorialAhorcado {
```

```
    string palabra;
    bool gano;
    int intentosRestantes;
    int puntos;
};

class Evaluacion {
private:
    float funcionalidad, defensa, estructura, innovacion;
public:
    Evaluacion() : funcionalidad(0), defensa(0), estructura(0), innovacion(0) {}

    bool setNotas(float f, float d, float e, float i) {
        if (f+d+e > 10.0) return false;
        funcionalidad = f; defensa = d; estructura = e; innovacion = i;
        return true;
    }
    float getSumaBase() const { return funcionalidad + defensa + estructura; }
    float getInnovacion() const { return innovacion; }
    float menorCriterio() const { return min({funcionalidad, defensa, estructura, innovacion}); }
};

class GrupoProyecto {
private:
    string codigo, tema, integrantes, observaciones;
    int avance, retrasos;
    float notaFinal;
    Evaluacion eval;

public:
    GrupoProyecto() : codigo(""), tema(""), integrantes(""), observaciones(""), avance(0), retrasos(0),
notaFinal(0) {}

    void registrar(string c, string t, string inst, int av, int ret, string obs) {
        codigo = c; tema = t; integrantes = inst; avance = av; retrasos = ret; observaciones = obs;
    }

    void setEvaluacion(Evaluacion e) {
        eval = e;
        notaFinal = eval.getSumaBase() + eval.getInnovacion() - (retrasos * 0.5f);
        if (notaFinal > 10) notaFinal = 10;
        if (notaFinal < 0) notaFinal = 0;
    }

    string getCodigo() const { return codigo; }
    string getTema() const { return tema; }
    float getNota() const { return notaFinal; }
    string getObs() const { return observaciones; }
    float getMenorCriterio() const { return eval.menorCriterio(); }

    void mostrarFila() {
        cout << codigo << "\t" << avance << "%\t" << notaFinal << "\t" << tema << endl;
    }
```

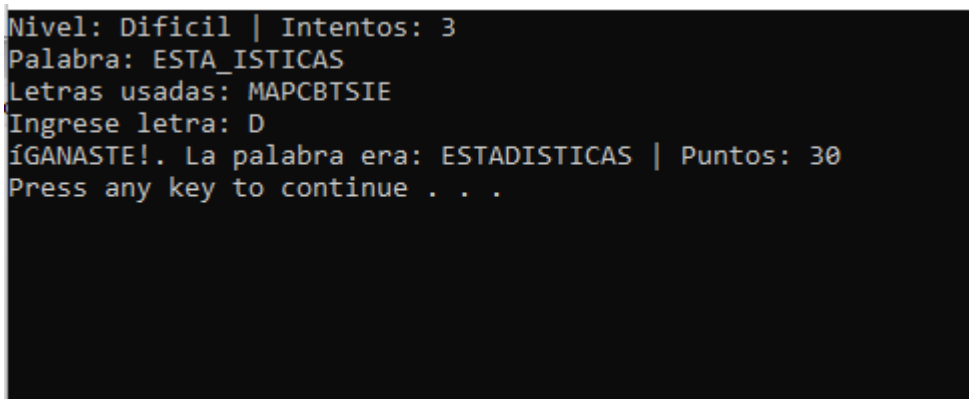
```
};

int validarEntero(string msg) {
    int v;
    while (true) {
        cout << msg;
        if (cin >> v) return v;
        cout << "Entrada invalida.\n";
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

float calcularPromedioRecursivo(GrupoProyecto grupos[], int n) {
    if (n <= 0) return 0;
    if (n == 1) return grupos[0].getNota();
    return (grupos[n-1].getNota() + (n-1) * calcularPromedioRecursivo(grupos, n-1)) / n;
}

void ordenarManual(GrupoProyecto arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j].getNota() < arr[j+1].getNota() ||
              (arr[j].getNota() == arr[j+1].getNota() && arr[j].getObs().length() > arr[j+1].getObs().length()))
{
                swap(arr[j], arr[j+1]);
            }
        }
    }
}

void buscarGrupo(GrupoProyecto grupos[], int cant) {
    string clave;
    cout << "Buscar por codigo o tema: "; cin.ignore(); getline(cin, clave);
    for(int i=0; i<cant; i++) {
        if(grupos[i].getCodigo() == clave || grupos[i].getTema() == clave) {
            cout << "Encontrado -> "; grupos[i].mostrarFila();
        }
    }
}

class ModuloAhorcado {
private:
    vector<HistorialAhorcado> historial;
    string facil[3] = {"GRUPO", "AVANCE", "TEMA"};
    string dificil[3] = {"INNOVACION", "PENALIZACION", "ESTADISTICAS"};
    int puntajeTotal = 0;

public:
    void mostrarMenu() {
        int op;
        do {
            system("cls");
            cout << "--- AHORCADO: GESTION DE PROYECTOS ---\n";
```

```
        cout << "1. Jugar (CPU)\n2. Ver Historial\n3. Ver Puntaje Total\n4. Volver\nOpcion: ";
        op = validarEntero("");
        if (op == 1) iniciarJuego();
        if (op == 2) verHistorial();
        if (op == 3) cout << "Puntaje acumulado: " << puntajeTotal << endl;
        if (op != 4) system("pause");
    } while (op != 4);
}

void iniciarJuego() {
    int nivel = validarEntero("Nivel (1. Facil, 2. Dificil): ");
    srand(time(0));
    string secreta = (nivel == 1) ? facil[rand()%3] : dificil[rand()%3];
    string progreso(secreta.length(), '_');
    int intentos = 6;
    string usadas = "";

    while (intentos > 0 && progreso != secreta) {
        system("cls");
        cout << "Nivel: " << ((nivel==1)?"Facil":"Dificil") << " | Intentos: " << intentos << endl;
        cout << "Palabra: " << progreso << "\nLetras usadas: " << usadas << endl;
        cout << "Ingrese letra: ";
        char letra; cin >> letra; letra = toupper(letra);

        if (!isalpha(letra) || usadas.find(letra) != string::npos) {
            cout << "Invalido o repetido.\n"; system("pause"); continue;
        }
        usadas += letra;

        if (secreta.find(letra) != string::npos) {
            for(int i=0; i<secreta.length(); i++) if(secreta[i]==letra) progreso[i]=letra;
        } else {
            intentos--;
            cout << "¡Incorrecto!\n";
        }
    }

    bool gano = (progreso == secreta);
    int puntos = gano ? intentos*10 : 0;
    puntajeTotal += puntos;
    cout << (gano ? "¡GANASTE!" : "PERDISTE") << ". La palabra era: " << secreta << " | Puntos: "
<< puntos << endl;
    historial.push_back({secreta, gano, intentos, puntos});
}

void verHistorial() {
    cout << "\n--- RESUMEN DE PARTIDAS ---\n";
    for(auto const& h : historial) {
        cout << "Palabra: " << h.palabra << " | Resultado: " << (h.gano?"Gano":"Perdio")
            << " | Puntos: " << h.puntos << endl;
    }
}
};
```

```
int main() {
   GrupoProyecto grupos[50];
   ModuloAhorcado juego;
   int cant = 0, op;

   do {
      system("cls");
      cout << "=== MENU ===\n";
      cout << "1. Registrar Grupo y Evaluacion\n2. Reporte (Ordenado por Nota)\n";
      cout << "3. Estadisticas (Promedio Recursivo)\n4. Buscar Grupo\n5. Criterio de menor
desempeño\n";
      cout << "6. Modulo Ahorcado\n7. Salir\nOpcion: ";
      op = validarEntero("");

      if (op == 1) {
         string c, t, inst, obs; int av, ret;
         cout << "Cod: "; cin >> c;
         cout << "Integrantes: "; cin.ignore(); getline(cin, inst);
         cout << "Tema: "; getline(cin, t);
         av = validarEntero("% Avance: ");
         ret = validarEntero("Retrasos: ");
         cout << "Observaciones: "; cin.ignore(); getline(cin, obs);

         grupos[cant].registrar(c, t, inst, av, ret, obs);

         Evaluacion e;
         float f, d, est, inn;
         cout << "Notas -> Funcionalidad: "; cin >> f;
         cout << "Defensa: "; cin >> d;
         cout << "Estructura:";
                  cout << "Estructura: "; cin >> est;
         cout << "Innovacion (Bono 0-2): "; cin >> inn;

         if(e.setNotas(f, d, est, inn)) {
            grupos[cant].setEvaluacion(e);
            cant++;
            cout << "Registro exitoso.\n";
         } else cout << "Error: Suma base supera 10.\n";
         system("pause");
      }
      else if (op == 2) {
         ordenarManual(grupos, cant);
         cout << "COD\tAVANCE\tNOTA\tTEMA\n";
         for(int i=0; i<cant; i++) grupos[i].mostrarFila();
         system("pause");
      }
      else if (op == 3) {
         if(cant > 0) cout << "Promedio General (Recursivo): " << calcularPromedioRecursivo(grupos,
cant) << endl;
         else cout << "No hay grupos registrados.\n";
         system("pause");
      }
```

```
    else if (op == 4) {
        buscarGrupo(grupos, cant);
        system("pause");
    }
    else if (op == 5) {
        if(cant > 0) {
            float menor = grupos[0].getMenorCriterio();
            for(int i=1; i<cant; i++) {
                menor = min(menor, grupos[i].getMenorCriterio());
            }
            cout << "El criterio de menor desempeño entre todos los grupos fue: " << menor << endl;
        } else {
            cout << "No hay grupos registrados.\n";
        }
        system("pause");
    }
    else if (op == 6) {
        juego.mostrarMenu();
    }
  } while (op != 7);

  return 0;
}
```

3.6 PRUEBA DE SOLUCION



4. Conclusiones

Orden y Seguridad: Al usar clases y objetos, el código quedó como un "archivo bien organizado". Esto permite que la información de los grupos y sus notas no se mezclen ni se borren por error, haciendo que el sistema sea más profesional.

Control Total: Como hicimos el ordenamiento (burbuja) y la búsqueda paso a paso sin usar funciones automáticas, demostramos que entendemos cómo se mueven los datos dentro de la memoria de la computadora.

Juego con Propósito: El Ahorcado no es solo un juego para perder el tiempo; logramos que se conecte con las notas. Si el estudiante gana, el sistema le da un premio real en su calificación, lo que une la diversión con el objetivo del programa.

Cálculos Inteligentes: Usar recursividad para el promedio general nos permitió resolver un problema matemático de una forma más corta y elegante, demostrando que podemos aplicar soluciones avanzadas en C++.

5. Recomendaciones

Evitar errores de dedo: Se recomienda mejorar las validaciones para que, si alguien escribe una letra por error donde va un número, el programa no se cierre de golpe, sino que pida el dato de nuevo amablemente.
Guardar el historial de cambios: Es importante seguir usando GitHub para guardar cada pequeño avance. Así, si algo se daña en el código, podemos "viajar en el tiempo" y recuperar lo que sí funcionaba.
Hacerlo crecer: Una buena idea sería que el programa avise automáticamente qué grupos están "en peligro" (por ejemplo, los que tienen menos de 7 de nota) para que el profesor pueda ayudarlos a tiempo.
Capacidad ilimitada: Por ahora el programa aguanta 50 grupos, pero a futuro sería mejor usar herramientas que permitan registrar todos los grupos que el usuario quiera sin límite de espacio.